Système d'exploitation
CM4 : Compétition, indéterminisme et synchronisation (1)

Pierre Rousselin
rousselin@univ-paris13.fr

Université Sorbonne Paris Nord
L2 informatique et double-licence

février 2026

Compétition et indéterminisme

Attente active (à éviter)

Appels système bloquants

Atomicité

Sémaphores

Compétition et indéterminisme

Attente active (à éviter)

Appels système bloquants

Atomicité

Sémaphores

Compétition : accès au terminal

Note : pour gagner de la place, à partir de maintenant on omet les inclusions de fichiers d'en-tête et certains tests de succès des appels systèmes (à faire tout de même !).

```
void *afficher_100_fois(void *arg);
int main(void)
{
    pthread_t th1, th2;
    pthread_create(&th1, NULL, afficher_100_fois, "A");
    pthread_create(&th2, NULL, afficher_100_fois, "B");
    pthread_join(th1, NULL);
    pthread_join(th2, NULL);
    printf("\n");
}
void *afficher_100_fois(void *arg)
{
    const char *ch = arg;
    for (int i = 0; i < 100; ++i)
        printf("%s", ch);
    return NULL;
}
```

Sortie ?

```
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAABAABABAAAABABBBBBABBAAAAAABAAAAA
AAAAAAAAAAAAABABBBBBAAAAAAAAAAAAABABBBBBABAAAAAAAAAAAAABABBBBBA
AAABAAAAAAABBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB
BBBBBBBBBBBBBBBB
```

- le résultat est imprévisible ;
- et change si l'on lance de nouveau programme...
- On parle *d'indéterminisme* lorsque le résultat d'une fonction ou d'un programme ne dépend pas uniquement de son entrée (arguments, fichiers ouverts en lecture, etc).

Situation de compétition

- On parle de situation de *compétition* (en anglais *race condition*)
- lorsque le résultat d'un programme
- ou d'un travail organisé (par exemple dans une usine, une cuisine, etc)
- diffère suivant l'ordre, arbitraire, dans lequel les acteurs (processus, *threads*, ouvriers, ...) accomplissent leurs tâches.

Situation de compétition

- On parle de situation de *compétition* (en anglais *race condition*)
- lorsque le résultat d'un programme
- ou d'un travail organisé (par exemple dans une usine, une cuisine, etc)
- diffère suivant l'ordre, arbitraire, dans lequel les acteurs (processus, *threads*, ouvriers, ...) accomplissent leurs tâches.

Exemple : yaourt à la fraise

Alice tant qu'il y a des pots de yaourt sans fraise, mettre de la confiture de fraise dans un pot de yaourt

Bob tant qu'il y a des pots de yaourt sans yaourt, mettre du yaourt dans un pot de yaourt

Situation de compétition

- On parle de situation de *compétition* (en anglais *race condition*)
- lorsque le résultat d'un programme
- ou d'un travail organisé (par exemple dans une usine, une cuisine, etc)
- diffère suivant l'ordre, arbitraire, dans lequel les acteurs (processus, *threads*, ouvriers, ...) accomplissent leurs tâches.

Exemple : yaourt à la fraise

Alice tant qu'il y a des pots de yaourt sans fraise, mettre de la confiture de fraise dans un pot de yaourt

Bob tant qu'il y a des pots de yaourt sans yaourt, mettre du yaourt dans un pot de yaourt

Rien n'impose que la confiture de fraise soit en-dessous.

Compétition et processus

Des processus entrent en compétition lorsqu'ils font des entrées/sorties sur les mêmes fichiers, plus précisément, si

- plusieurs processus écrivent dans le même fichier ou bien
- au moins un processus écrit dans un fichier et d'autres le lisent.

Compétition et processus

Des processus entrent en compétition lorsqu'ils font des entrées/sorties sur les mêmes fichiers, plus précisément, si

- plusieurs processus écrivent dans le même fichier ou bien
- au moins un processus écrit dans un fichier et d'autres le lisent.

Exemple de compétition en écriture dans le même fichier (trois shells écrivent dans le terminal) :

```
$ echo A & echo B &
[9] 32285
A
[10] 32286
B
$
```

Compétition et processus

Des processus entrent en compétition lorsqu'ils font des entrées/sorties sur les mêmes fichiers, plus précisément, si

- plusieurs processus écrivent dans le même fichier ou bien
- au moins un processus écrit dans un fichier et d'autres le lisent.

Exemple de compétition en écriture dans le même fichier (trois shells écrivent dans le terminal) :

```
$ echo A & echo B &
[9] 32285
A
[10] 32286
B
$
```

mais aussi :

```
$ echo A & echo B &
[1] 32290
[2] 32291
A
$ B
```

(et tous les autres cas sont possibles en théorie).

Compétition entre *threads*

Entre *threads* c'est encore pire car la mémoire est partagée ! En plus des entrées sorties, on a aussi compétition lorsque plusieurs threads accèdent à une même case de la mémoire, avec au moins l'un d'entre eux qui modifie son contenu.
Exemple :

- Alice fait des choses puis écrit (produit) dans une variable globale `resultat` un caractère ;
- Bob fait des choses puis lit (utilise, consomme) dans `resultat` le caractère et l'affiche.

Deux cas peuvent survenir :

- Alice produit avant que Bob consomme, ok.
- Bob consomme avant qu'Alice produise, ko.

Alice et Bob

```
char res;
int main(void)
{
    pthread_t a, b;
    srand(time(NULL));
    pthread_create(&a, NULL, alice, NULL);
    pthread_create(&b, NULL, bob, NULL);

    pthread_join(a, NULL);
    pthread_join(b, NULL);
    return 0;
}
void faire_des_choses(void) { /* ... */ }
void *alice(void *arg) { faire_des_choses(); res = 'A'; }
void *bob(void *arg) { faire_des_choses(); printf("%c\n", res); }
```

Sortie (possible)

Alice et Bob

```c
char res;
int main(void)
{
    pthread_t a, b;
    srand(time(NULL));
    pthread_create(&a, NULL, alice, NULL);
    pthread_create(&b, NULL, bob, NULL);

    pthread_join(a, NULL);
    pthread_join(b, NULL);
    return 0;
}
void faire_des_choses(void) { /* ... */ }
void *alice(void *arg) { faire_des_choses(); res = 'A'; }
void *bob(void *arg) { faire_des_choses(); printf("%c\n", res); }
```

Sortie (possible)

```
$ gcc -Wall alice_bob1.c -pthread
$ ./a.out

$ ./a.out
A
```

Compétition et indéterminisme

Attente active (à éviter)

Appels système bloquants

Atomicité

Sémaphores

Bob a une idée

```
int alice_a_fini = 0;
int main(void)
{
    /* comme avant */
}
void faire_des_choses() { /* ... */ }
void *alice(void *arg) {
    faire_des_choses();
    res = 'A';
    alice_a_fini = 1;
}
void *bob(void *arg) {
    faire_des_choses();
    while (!alice_a_fini)
        ;
    printf("%c\n", res);
}
```

Problème résolu ?

Ce n'est pas idiot (et vraiment le début de quelque chose).

Problème résolu ?

Ce n'est pas idiot (et vraiment le début de quelque chose). Mais regardons le code en assembleur produit :

```
.L6:
        movl        alice_a_fini(%rip), %eax
        testl        %eax, %eax
        je          .L6
```

Problème résolu ?

Ce n'est pas idiot (et vraiment le début de quelque chose). Mais regardons le code en assembleur produit :

```
.L6:
        movl        alice_a_fini(%rip), %eax
        testl        %eax, %eax
        je          .L6
```

Pendant que Bob attend qu'Alice ait fini, le processeur :

- charge en mémoire le contenu de `alice_a_fini` ;
- compare son contenu à `0`
- si oui saute deux lignes plus haut
- sinon continue l'exécution

Problème résolu ?

Ce n'est pas idiot (et vraiment le début de quelque chose). Mais regardons le code en assembleur produit :

```
.L6:
        movl        alice_a_fini(%rip), %eax
        testl        %eax, %eax
        je          .L6
```

Pendant que Bob attend qu'Alice ait fini, le processeur :

- charge en mémoire le contenu de `alice_a_fini` ;
- compare son contenu à `0`
- si oui saute deux lignes plus haut
- sinon continue l'exécution

Si Alice met un millième de seconde de plus que Bob à faire ses choses, un processeur qui a une fréquence d'horloge d'1 GHz fait un million d'opérations, 333333 fois ces trois opérations.

Problème résolu ?

Ce n'est pas idiot (et vraiment le début de quelque chose). Mais regardons le code en assembleur produit :

```
.L6:
        movl        alice_a_fini(%rip), %eax
        testl        %eax, %eax
        je          .L6
```

Pendant que Bob attend qu'Alice ait fini, le processeur :

- charge en mémoire le contenu de `alice_a_fini` ;
- compare son contenu à `0`
- si oui saute deux lignes plus haut
- sinon continue l'exécution

Si Alice met un millième de seconde de plus que Bob à faire ses choses, un processeur qui a une fréquence d'horloge d'1 GHz fait un million d'opérations, 333333 fois ces trois opérations.
Ça réchauffe la planète... et empêche le processeur de faire autre chose de plus utile. Ce procédé, appelé **attente active** est à éviter !

Compétition et indéterminisme

Attente active (à éviter)

Appels système bloquants

Atomicité

Sémaphores

Appels système bloquant

C'est là qu'intervient notre meilleur ami, le noyau.

Appels système bloquant

C'est là qu'intervient notre meilleur ami, le noyau. Un appel système est dit bloquant lorsqu'il est associé à une condition et

- retourne immédiatement lorsque la condition est remplie ; ou bien
- met en sommeil la tâche qui fait cet appel si la condition n'est pas réalisée ...
- puis réveille cette tâche lorsque la condition est satisfaite.

Appels système bloquant

C'est là qu'intervient notre meilleur ami, le noyau. Un appel système est dit bloquant lorsqu'il est associé à une condition et

- retourne immédiatement lorsque la condition est remplie ; ou bien
- met en sommeil la tâche qui fait cet appel si la condition n'est pas réalisée ...
- puis réveille cette tâche lorsque la condition est satisfaite.

Exemples (certains déjà rencontrés) :

- `sleep`, `usleep`, `nanosleep` : dormir en attendant qu'une certaine durée soit écoulée
- `wait` : si un enfant a déjà fini, retourne immédiatement, sinon attend
- `read` dans un `pipe` ou une `socket` : s'il y a des octets à lire, les lire et retourner le nombre d'octets lus, sinon attendre qu'il y en ait
- `pthread_join` : attendre la terminaison d'un thread

join

Ici, la solution est toute trouvée :

- le thread initial transmet à `bob` l'identifiant de *thread* d'`alice` ;
- `bob` attend qu'`alice` ait fini avec `pthread_join`.

```
int main(void)
{
    pthread_t a, b;
    pthread_create(&a, NULL, alice, NULL);
    pthread_create(&b, NULL, bob, &a);

    pthread_join(b, NULL);
    return 0;
}
void *bob(void *thread)
{
    pthread_t *a = thread;
    faire_des_choses();
    pthread_join(*a, NULL);
    printf("%c\n", res);
    return NULL;
}
```

Résultat

```
bob:
.LFB9:
        ; ...
        call        pthread_join
        ; ...
```

Version avec attente active de Bob : voir champ `user`.

```
$ time ./a.out
A

real  0m0,302s
user  0m0,051s
sys   0m0,001s
```

Même chose avec `join` (bloquant) :

```
$ time ./a.out
A

real  0m0,303s
user  0m0,000s
sys   0m0,003s
```

Nouveau problème

Si `alice` a encore des choses à faire après avoir écrit ?

- Besoin d'autres appels bloquants.
- Dans ce cours : mutex et sémaphores.

Compétition et indéterminisme

Attente active (à éviter)

Appels système bloquants

Atomicité

Sémaphores

$1000000 = 1000 \times 1000$

```
#define N 1000
void *bosser(void *);
int incremente_moi = 0;
int main(void)
{
    pthread_t th[N];
    int i;
    for (i = 0; i < N; ++i)
        pthread_create(&th[i], NULL, bosser, NULL);
    for (i = 0; i < N; ++i)
        pthread_join(th[i], NULL);

    printf("incremente_moi = %d\n", incremente_moi);
    return 0;
}
void *bosser(void *arg)
{
    int i;
    for (i = 0; i < N; ++i)
        ++incremente_moi;
    return NULL;
}
```

Exécution

```
$ gcc -Wall million.c -pthread
$ ./a.out
incremente_moi = 997592
$ ./a.out
incremente_moi = 996979
$ ./a.out
incremente_moi = 997493
```

Angoissant...

Incrémenter

Code assembleur (x86-64) :

```
movl        incremente_moi(%rip), %eax
addl        $1, %eax
movl        %eax, incremente_moi(%rip)
```

Incrémenter

Code assembleur (x86-64) :

```
movl        incremente_moi(%rip), %eax
addl        $1, %eax
movl        %eax, incremente_moi(%rip)
```

Une ligne de code en C, mais 3 instructions :

- Copier la valeur de la globale `incremente_moi` dans le registre `%eax` ;
- ajouter 1 au contenu du registre `%eax` ;
- copier le contenu de `%eax` dans la globale `incremente_moi`.

On dit que l'incrémentation n'est pas une instruction atomique. En général l'atomicité dépend

- du compilateur (voire du niveau d'optimisation de celui-ci) ;
- de l'architecture du processeur.

Problème de non-atomicité

Où est le problème avec ?

```
movl        incremente_moi(%rip), %eax
addl        $1, %eax
movl        %eax, incremente_moi(%rip)
```

Une possibilité, avec deux *threads* `t1` et `t2` sur un seul processeur qui incrémentent tous les deux `incremente_moi` :

1. `t1` charge `incremente_moi` qui vaut, disons, 100, dans le registre `eax`, puis incrémente le contenu d'`eax` qui contient 101.
2. PAF ! Commutation de contexte, le système donne la main à `t2`. Tous les registres associés à `t1` sont sauvegardés.
3. `t2` charge `incremente_moi` qui vaut toujours 100, dans le registre `eax`, puis incrémente le contenu d'`eax` qui contient 101 et met le contenu de `eax` (101) dans `incremente_moi`.
4. Le système redonne la main à `t1`, restore l'état dans lequel était le processeur au moment de la commutation de contexte : contenu de `eax` et compteur de programme.
5. `t1` reprend où il en était, il charge la valeur contenue dans `eax` (101) dans `incremente_moi`...
6. ... qui n'a donc été incrémenté qu'une seule fois au lieu de 2.

Vu en TP, à ne pas faire

Parfois des *threads* ont besoin d'un indice pour fonctionner.

```
#define N 100
void *print_i(void *addr_i);
int main(void)
{
        pthread_t th[N];
        int i;
        for (i = 0; i < N; ++i)
                pthread_create(th + i, NULL, print_i, &i);
        for (i = 0; i < N; ++i)
                pthread_join(th[i], NULL);
        return 0;
}

void *print_i(void *addr_i)
{
        int i = *((int *) addr_i);
        printf("i = %d\n", i);
        return NULL;
}
```

Compteur local à `main` et *threads*

```
$ gcc -Wall threads_compteur.c -pthread
$ ./a.out | sort -u | wc -l
95
$ ./a.out | sort -u | wc -l
94
$ ./a.out | sort -u | wc -l
96
```

La valeur de i peut changer entre l'appel à `pthread_create` et le déréférencement dans `print_i` !

Une solution possible

```
#define N 100
void *print_i(void *addr_i);
int main(void)
{
    pthread_t th[N];
    int cpt[N];
    int i;
    for (i = 0; i < N; ++i) {
        cpt[i] = i;
        pthread_create(th + i, NULL, print_i, cpt + i);
    }
    for (i = 0; i < N; ++i)
        pthread_join(th[i], NULL);
    return 0;
}
void *print_i(void *addr_i)
{
    int i = *((int *) addr_i);
    printf("i = %d\n", i);
    return NULL;
}
```

Compteur et *threads*, solution

- Le contenu de `cpt[i]` n'est pas modifié après son initialisation.
-
```
$ gcc -Wall threads_compteur2.c -pthread
$ ./a.out | sort -u | wc -l
100
$ ./a.out | sort -u | wc -l
100
$ ./a.out | sort -u | wc -l
100
```
- technique courante en calcul distribué : la promotion d'un scalaire (une variable numérique) en un vecteur (un tableau).

Un autre problème

Considérer aussi :

- Dans un premier thread :

```
n = 3;
/* ... */
n = 0;
```

- Dans un second thread :

```
if (n != 0) {
    x = 8 / n;
    /* ... */
}
```

- Possible que :
  1. le premier thread met 3 dans `n` ;
  2. le second fait le test `n != 0` puis saute à l'instruction suivante ;
  3. le premier thread met `0` dans `n` ;
  4. le second thread calcule `8 / 0` : arrêt du programme avec `Floating Point Exception`

Compétition et indéterminisme

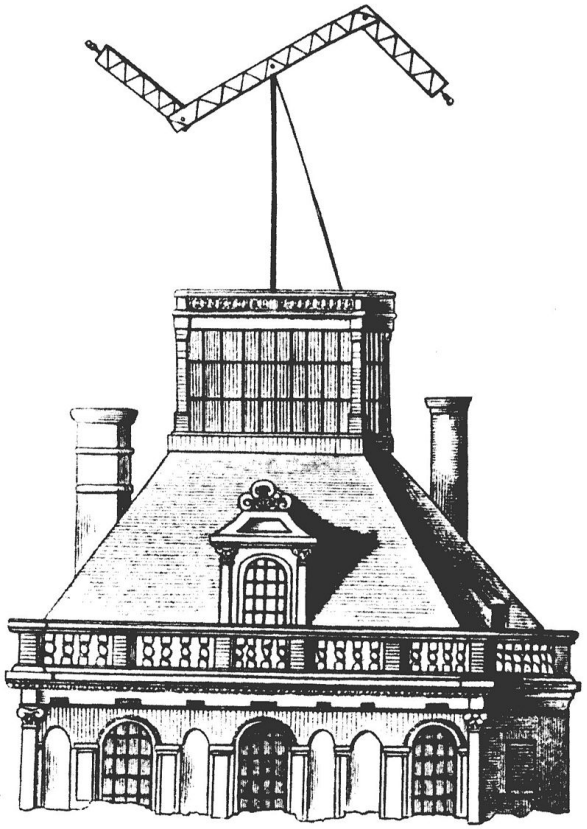
Attente active (à éviter)

Appels système bloquants

Atomicité

Sémaphores

Sémaphores



Sémaphores

- Du grec *sema* signe et *phoros*, porter.
- Inventés par Dijkstra.
- Objet opaque contenant un compteur ≥ 0.
- Un compteur à 0 peut s'interpréter comme une absence de ressource.
- Un compteur à $n > 0$ peut s'interpréter comme la présence de $n > 0$ ressources qui vont pouvoir être consommées.
- Ce compteur n'est jamais lu ni modifié directement.

Sémaphores

Seulement 4 opérations (dans ce cours et dans presque tous les cours sur les sémaphores).

```
#include <semaphore.h>
sem_init(sem_t *s, 0, val);
sem_wait(sem_t *s);
sem_post(sem_t *s);
sem_destroy(sem_t *s);
```

Initialisation et destruction

```
int sem_init(sem_t *s, 0, unsigned val);
```

- Initialise le sémaphore d'adresse `s`.
- Comme d'habitude, valeur de retour 0 si succès $\neq 0$ si échec.
- Deuxième argument : pour utiliser le sémaphore avec des processus, on oublie, toujours 0.
- Troisième argument : valeur initiale du sémaphore :
  - $n > 0$ si on considère qu'il doit être initialement « ouvert », ou encore qu'on commence avec $n > 0$ ressource disponible ;
  - 0 sinon (fermé, bloqué, pas de ressource, ...)
- Le choix de cette valeur initiale est critique.

```
int sem_destroy(sem_t *sem);
```

pour détruire le sémaphore lorsqu'on n'en a plus besoin (libération de ressources).

Attente d'une condition/Consommation d'une ressource

```
int sem_wait(sem_t *s);
```

Un *thread* dont l'exécution arrive à cette instruction :

- est *endormi* si le compteur associé au séminaire d'adresse `s` est 0
- ou bien décrémente ce compteur puis passe à la prochaine instruction s'il est > 0.
- Opération atomique.

Signaler une production ou la réalisation d'une condition

```
int sem_post(sem_t *s);
```

- Incrémente le compteur associé au sémaphore d'adresse `s` ;
- Si des *threads* sont endormis en attente sur le sémaphore d'adresse `s`, réveille un et un seul d'entre eux (qui décrémentera alors immédiatement le compteur).
- Opération atomique.

Pour les heureux (?) possesseurs de Mac

Les sémaphores anonymes (ceux décrits plus haut) ne sont pas implémentés dans MacOS (en fait l'appel `sem_init` se contente de toujours retourner une erreur...)
Heureusement, il y a tout de même les sémaphores nommés qui s'utilisent presque de la même manière :

```
sem_t *mon_sem;
if ((mon_sem = sem_open("/mon_sem", O_CREAT | O_EXCL, 0600, 1)
        == SEM_FAILED) {
    perror("creation mon_sem");
    exit(1);
}

/* utiliser le sémaphore comme d'habitude */
sem_close(mon_sem); /* fermer le  fichier */
sem_unlink("/mon_sem"); /* supprimer le fichier */
```

Alice et Bob au pays des sémaphores

```
void *alice(void *);
void *bob(void *);
void faire_des_choses();
char res;
int main(void)
{
    pthread_t a, b;
    sem_t s;
    sem_init(&s, 0, 0); /* initialement, 0 ressource */
    srand(time(NULL));
    pthread_create(&a, NULL, alice, &s);
    pthread_create(&b, NULL, bob, &s);

    pthread_join(b, NULL);
    pthread_join(a, NULL);

    sem_destroy(&s);
    return 0;
}
void faire_des_choses() { /* ... */ }
```

Alice et Bob au pays des sémaphores (2)

```
void *alice(void *sem)
{
    sem_t *s = sem;
    faire_des_choses();
    res = 'A';
    sem_post(s); /* dire à Bob qu'il peut continuer */
    faire_des_choses();
    printf("Alice : j'ai fait pas mal de choses !\n");
    return NULL;
}
void *bob(void *sem)
{
    sem_t *s = sem;
    faire_des_choses();
    /* bloque si besoin jusqu'à ce qu'Alice ait fini */
    sem_wait(s);
    printf("Bob : Alice a fini et a écrit : %c\n", res);
    return NULL;
}
```

Plus compliqué

- Alice fait un certain nombre de calculs (elle produit des valeurs).
- Lorsqu'un tel résultat est prêt, Bob l'utilise (ici, l'affiche).
- Lorsque tous les résultats ont été produit, Alice doit communiquer à Bob qu'il n'y a plus rien à produire.
- Bob doit attendre qu'Alice produise `res` avant de l'utiliser.
- Alice doit attendre avant de mettre à jour `res` que Bob l'ait utilisé.

Deux sémaphores et une globale

- On va utiliser un tableau `s` de deux sémaphores :
  - le premier élément (`s[0]`) est utilisé par Alice pour dire qu'elle a fini de produire un résultat
  - le second (`s[1]`) par Bob pour dire qu'il a fini d'utiliser le résultat précédent
- une variable globale `fini` est mise à 1 lorsqu'Alice a fini toutes ses productions.

Nouvelle fonction main

```
char res = 'A' - 1;
int fini = 0;
int main(void)
{
    pthread_t a, b;
    sem_t s[2];
    /* initialement alice n'a pas encore produit */
    sem_init(s, 0, 0);
    /* bob est prêt à afficher une nouvelle valeur */
    sem_init(s + 1, 0, 1);
    srand(time(NULL));
    pthread_create(&a, NULL, alice, s);
    pthread_create(&b, NULL, bob, s);

    pthread_join(b, NULL);
    pthread_join(a, NULL);

    sem_destroy(s);
    sem_destroy(s + 1);
    return 0;
}
```

Nouveau code d'Alice

```
void *alice(void *sem)
{
    sem_t *s = sem;
    while (res < 'Z') {
        faire_des_choses();
        sem_wait(s + 1);
        ++res;
        if (res == 'Z')
            fini = 1;
        sem_post(s);
    }
    printf("Alice: j'ai fait pas mal de choses !\n");
    return NULL;
}
```

Nouveau code de Bob

```
void *bob(void *sem)
{
    sem_t *s = sem;
    for (;;) {
        faire_des_choses();
        sem_wait(s);
        printf("Bob: Alice a fini et a écrit: %c\n", res);
        if (fini)
            break;
        sem_post(s + 1);
    }
    return NULL;
}
```